Algoritmos e Programação: Fundamentos

Vetores e Strings

Profa. Andriele Busatto do Carmo

acarmo@unisinos.br

UNISINOS

Vetores

Vetores

- Coleção de **variáveis de mesmo tipo**.

- Coleção acessada por um **único nome**. A **individualização do acesso a cada variável é feita por meio de índices**.

- Estrutura com uma única dimensão.

Declaração

- Sintaxe: **tipo nome[tamanho];**

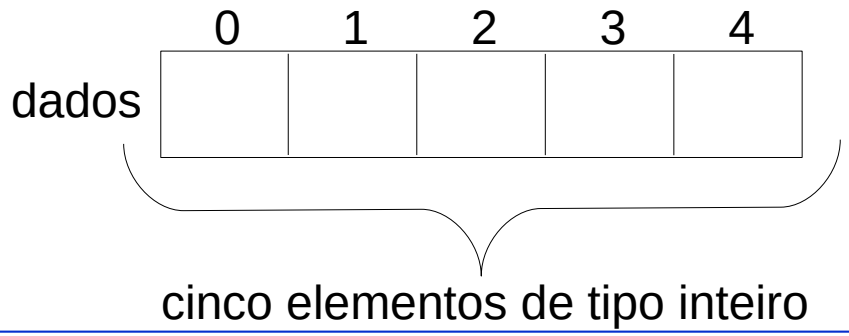
Tipo dos dados
Qualquer tipo válido

Nome válido

Quantidade máxima de elementos

Exemplo:

`int dados[5];` → declaração de um vetor com cinco posições de tipo inteiro.



Acesso aos elementos

- Para acessar os elementos de um vetor, é necessário o uso dos índices.
  - O índice define a posição da variável/elemento no vetor.

- O primeiro elemento estará sempre na posição de índice zero.

- O último elemento, considerando um vetor de tamanho N, estará sempre na posição N-1.

Exemplos

```
int vetor[3];  // declara um vetor de 5 posições



vetor[0] = 4;  // insere 4 na primeira posição do vetor

vetor[2] = 15; // insere 15 na última posição do vetor
```

Exemplos

- Preenchimento do vetor com o mesmo dado em todas as posições:

```
for(i = 0; i < 3; i++) {
   vetor[i] = 10;
}
```

- Inserir os números de 1 a 3 no vetor previamente declarado:

```
for(i = 0; i < 3; i++) {
   vetor[i] = i + 1;
}
```

Exemplos

- Inserir os números de 3 a 1 no vetor previamente declarado:

```
for(i = 0; i < 3; i++) {
   vetor[i] = 3 – i;
}
```

- Uso de constantes para definir o tamanho de um vetor:

```
#define TAM_MAX 10

float valores[TAM_MAX];

for(i = 0; i < TAM_MAX; i++) {
   valores[i] = TAM_MAX – i;
}
```

Exemplos

- Leitura/entrada de dados em um vetor:

```
#define TAM_MAX 10

float exemplo[TAM_MAX];

for(i = 0; i < TAM_MAX; i++) {
   scanf("%f", &exemplo[i]);
}
```

Passagem de vetor como parâmetro

- Para passar um vetor como parâmetro, basta declarar o parâmetro **da mesma forma** que o vetor foi declarado.

- Na linguagem C, por definição, **um vetor é sempre passado como referência**.
  - Neste caso, qualquer alteração em seus elementos, altera a variável usada como parâmetro na chamada da rotina.

Passagem de vetor como parâmetro

- Exemplo:

```
#include <stdio.h>

#define TAM_MAX 10


void imprime_vetor(int tamanho, int vetor[TAM_MAX]) {

     int i;

     for (i = 0; i < tamanho; i++) {

          printf("%d\n", vetor[i]);

     }

}


int main() {

     int i, n[TAM_MAX];

     for (i = 0; i < TAM_MAX; i++) {

          n[i] = i + 1;

     }

     imprime_vetor(TAM_MAX, n); // Passa o vetor 'n' como parâmetro

     return 0;

}
```

Limites dos índices de vetores

- A linguagem C não faz nenhum teste de verificação dos índices usados para acessar os elementos de um vetor.

- Se estes limites não forem respeitados, resultados indesejados serão obtidos.

Limites dos índices de vetores

- Exemplo:

```
#include <stdio.h>

int main() {
    int vetor[3];
    int i;

    for(i = 0; i < 10; i++)
        vetor[i] = i; // o vetor tem 3 elementos,
                      // mas o laço do for vai até 10

    for(i = 0; i < 3; i++)
        printf("%d ", vetor[i]);

    return 0;
}
```

Passagem de elementos do vetor por parâmetro (por valor)

- É realizada da mesma forma que a passagem de uma variável comum.

- Exemplo:

```
#include <stdio.h>
#include <string.h>
void imprime (int n) // parâmetro passado por valor
{
     printf("%d", n);
}
```

```
int main() {
     int i;
     int vetor[10];

     for(i = 0; i < 10; i++)
          vetor[i] = i;

     for (i = 0; i < 10; i++)
           imprime(vetor[i]); // passa o valor
                              // de cada um dos
                              // elementos do vetor
                              // como parâmetro

     return 0;
}
```

Passagem de elementos do vetor por parâmetro (por referência)

- É realizada da mesma forma que a passagem de uma variável.
  - Logo, se para a passagem por valor usa-se `vetor[i]`, na passagem por referência usa-se `&vet[i]`.

Passagem de elementos do vetor por parâmetro (por referência)

- Exemplo:

```
#include <stdio.h>
#include <string.h>

// função com um parâmetro por valor
void imprime (int n) {
      printf("%d", N);
}

// função com um parâmetro por referência
void atualiza (int *n, int i) {
      *n = i;
}
```

```
int main() {
      int i;
      int vetor[10];

      for(i = 0; i < 10; i++) {
            atualiza(&vetor[i], i); // Note que é preciso colocar o '&' antes
                                    // de vetor[i] pois a função 'atualiza'
                                    // espera um parâmetro por REFERÊNCIA

            imprime(vetor[i]); // Note que NÃO se deve colocar nada antes
                               // de vetor[i] pois a função 'imprime' espera um
                               // parâmetro por VALOR
      }

      return 0;
}
```

Strings

Strings

- Uma string é uma sequência de caracteres.
  - Utiliza-se aspas duplas para representá-la no código.
  - Exemplo: “Este é um exemplo de string”.

Strings em C

- A linguagem C não possui o tipo primitivo **string**.

- Sequência de caracteres em C são **representadas por vetores de tipo char** terminadas, obrigatoriamente, pelo caractere nulo '**\0**' (barra inversa e zero).
  - Por este motivo, é necessário reservar uma posição na sequência para inserir o caractere nulo.

Strings em C

- Exemplos:

```
char cidade[13] = {‘P’, ‘o’, ‘r’, ‘t’, ‘o’, ‘ ’,
                   ‘A’, ‘l’, ‘e’, ‘g’, ‘r’, ‘e’, ‘\0’};
char palavra[30] = {‘A’, ‘l’, ‘g’, ‘o’, ‘r’, ‘i’, ‘t’, ‘m’, ‘o’, ‘\0’};
```

- Equivalente a:

```
char cidade[13] = “Porto Alegre”;
char palavra[30] = “Algoritmo”;
```

Strings em C

- Diferentes maneiras de declarar e inicializar strings:

```
char s1[] = ""; //aspas duplas sem espaços entre elas
char s2[] = "Porto Alegre";
char s3[50];
char s4[50] = "Porto Alegre";
```

  - s1 declara uma string vazia. Possui um único caractere: '\0'.
  - s2 declara uma string com 13 caracteres.
  - s3 declara uma string com até 49 caracteres que não foi inicializada. Lembrar que a última posição é o '\0'.
  - s4 declara uma string com até 49 caracteres. Neste exemplo a string foi inicializada.

Funções para manipulação de strings em C

- Função **`fgets`**
  - Sintaxe: **`fgets(char *str, int num, FILE *stream);`**
  - Lê **`num-1`** caracteres de **`stream`** e coloca-os no vetor de caracteres **`str`**.

Funções para manipulação de strings em C

- Função **`fgets`**
  - Exemplo:

```
#include <stdio.h>

int main() {
    char s[50];

    fgets(s, 50, stdin); //lê da entrada padrão
    printf("%s", s);

    return 0;
}
```

Funções para manipulação de strings em C

- Função **`fputs`**
  - Sintaxe: **`fputs(const char *str, FILE *stream);`**
  - Escreve na **`stream`** especificada o conteúdo da string apontada por **`str`**. O caractere nulo não é escrito.
  - Exemplo:

```
#include <stdio.h>

int main() {
    fputs(“Aula de Algoritmos”, stdout);

    return 0;
}
```

Funções para manipulação de strings em C

- Função **`strcpy`**
  - Sintaxe: **`strcpy(char *str1, const char *str2);`**
  - Copia o conteúdo da string **`str2`** para a string **`str1`**.
  - Incluir o cabeçalho da biblioteca string:

```
#include <string.h>
```

Funções para manipulação de strings em C

- Função **`strcpy`**
  - Exemplo:

```
#include <stdio.h>
#include <string.h>

int main() {
   char s[50];

   strcpy(s, “Aula de algoritmos”);
   fputs(s, stdout);

   return 0;
}
```

Funções para manipulação de strings em C

- Função **`strcat`**
  - Sintaxe: **`strcat(char *str1, const char *str2);`**
  - Concatena uma cópia de **`str2`** em **`str1`** e termina **`str1`** com um caractere nulo.
  - Incluir o cabeçalho da biblioteca string:

```
#include <string.h>
```

Funções para manipulação de strings em C

- Função **`strcat`**
  - Exemplo:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[20], s2[10];

    strcpy(s1, “Boa”);
    strcpy(s2, “Noite!”);
    strcat(s1, s2); // concatena em s1 a string s2
    printf(“%s\n”, s1)

    return 0;
}
```

Funções para manipulação de strings em C

- Função **`strcmp`**
  - Sintaxe: **`strcmp(const char *str1, const char *str2);`**
  - Compara lexicograficamente duas strings e retorna um inteiro baseado no resultado.
    - Retorna 0 se **`str1`** é igual a **`str2`**, um valor menor do que zero se **`str1`** é menor que **`str2`**, e um valor maior que zero se **`str1`** é maior que **`str2`**.
  - Incluir o cabeçalho da biblioteca string:

```
#include <string.h>
```

Funções para manipulação de strings em C

- Função **`strcmp`**
  - Exemplo:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[30];

    printf(“Informe a sua senha: “);
    fgets(s, 30, stdin);
    if(strcmp(s, “laranja”))
        printf(“Senha inválida\n”);
    else
        printf(“Senha ok\n”);

    return 0;
}
```

Funções para manipulação de strings em C

- Função **`strlen`**
  - Sintaxe: **`strlen(const char *str);`**
  - Retorna o comprimento da string terminada por um caractere nulo apontada para **`str`**.
  - Incluir o cabeçalho da biblioteca string:

```
#include <string.h>
```

Funções para manipulação de strings em C

- Função **`strlen`**
  - Exemplo:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[20] = “Algoritmos”;
    int tamanho;

    tamanho = strlen(s);
    printf(“O tamanho da string %s é: %d\n”, s, tamanho);

    return 0;
}
```

Referência Bibliográfica

- SCHILDT, Herbert. C Completo e Total. 3ª edição. Makron Books.